

TUGAS PRAKTIKUM 4
Sinkronisasi Sistem Operasi



Dian Chandra M
2410651032

UNIVERITAS MUHAMMDIYAH JEMBER
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INFORMATIKA
2025

SISTEM BOOKING HOTEL (Akhiran NIM Ganjil)

Deskripsi Masalah:

- Anda diminta membuat simulasi sistem booking hotel dengan kondisi:
- Hotel memiliki **5 kamar** yang bisa di-booking
- Ada **10 customer** yang mencoba booking secara bersamaan (10 threads)
Setiap customer akan:
 1. Mencoba booking kamar (butuh waktu random 1-3 detik)
 2. Jika berhasil, "menginap" selama 5 detik
 3. Check-out dan melepaskan kamar
 Jika gagal (kamar penuh), customer menunggu 2 detik lalu coba lagi

Customer **maksimal mencoba 3 kali**, jika gagal terus maka menyerah

Kode Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

// --- KONSTANTA ---
#define TOTAL_ROOMS 5
#define TOTAL_CUSTOMERS 10
#define MAX_ATTEMPTS 3
#define SLEEP_BOOKING_MIN 1
#define SLEEP_BOOKING_MAX 3
#define SLEEP_STAY 5
#define SLEEP_RETRY 2

// --- VARIABEL GLOBAL ---
sem_t room_semaphore; // Semaphore untuk kamar yang tersedia
pthread_mutex_t print_mutex; // Mutex untuk output dan perubahan status kamar
int room_status[TOTAL_ROOMS]; // 0 = Kosong, 1 = Terisi. (Dilindungi oleh print_mutex saat diakses)

// --- FUNGSI UTILITY ---

// Fungsi untuk mendapatkan timestamp dalam format [HH:MM:SS]
char* get_timestamp() {
    time_t timer;
    struct tm* tm_info;

    // Alokasi statis agar nilainya tetap ada saat dipanggil
    static char buffer[12];
```

♦

```

time(&timer);
tm_info = localtime(&timer);

// Format waktu menjadi [HH:MM:SS]
strftime(buffer, 12, "[%H:%M:%S]", tm_info);
return buffer;
}

// Mencari kamar kosong pertama dan mengembalikannya (atau -1 jika penuh)
// Asumsi: Dipanggil dalam lock Mutex
int find_available_room() {
    for (int i = 0; i < TOTAL_ROOMS; i++) {
        if (room_status[i] == 0) {
            room_status[i] = 1; // Tandai sebagai terisi
            return i + 1; // Nomor kamar (1-based index)
        }
    }
    return -1;
}

// Menghitung kamar yang tersedia saat ini
int get_available_count() {
    int count;
    sem_getvalue(&room_semaphore, &count);
    return count;
}

// --- FUNGSI THREAD CUSTOMER ---
void *customer_action(void *arg) {
    int customer_id = *(int *)arg;
    int attempts = 0;
    int room_number = -1;

    while (attempts < MAX_ATTEMPTS) {
        attempts++;

        // 1. Mencoba booking kamar
        // Simulasi waktu proses booking
        int booking_time = (rand() % (SLEEP_BOOKING_MAX - SLEEP_BOOKING_MIN + 1)) +
        SLEEP_BOOKING_MIN;

        // Log saat mencoba booking
        pthread_mutex_lock(&print_mutex);
        printf("%s 🧑 Customer %02d mencoba booking...\n", get_timestamp(), customer_id);
        pthread_mutex_unlock(&print_mutex);

        usleep(booking_time * 1000000); // Tunggu selama waktu booking

        // Coba ambil semaphore (kamar)
        if (sem_trywait(&room_semaphore) == 0) {
            // --- BERHASIL BOOKING ---

            // Ambil kamar di bawah perlindungan Mutex
            pthread_mutex_lock(&print_mutex);

```

```

    room_number = find_available_room(); // Ambil nomor kamar dan tandai terisi

    printf("%s ✅ Customer %02d berhasil booking kamar %d! (%d kamar tersisa)\n",
           get_timestamp(), customer_id, room_number, get_available_count());

    pthread_mutex_unlock(&print_mutex);

    // 2. "Menginap" selama 5 detik
    usleep(SLEEP_STAY * 1000000);

    // 3. Check-out dan melepaskan kamar
    pthread_mutex_lock(&print_mutex);

    // Lepaskan kamar dari status
    if (room_number != -1) {
        room_status[room_number - 1] = 0; // Tandai kamar sebagai kosong
    }

    // Lepaskan semaphore (kamar)
    sem_post(&room_semaphore);

    printf("%s 📄 Customer %02d check-out dari kamar %d (%d kamar tersisa)\n",
           get_timestamp(), customer_id, room_number, get_available_count());

    pthread_mutex_unlock(&print_mutex);

    return NULL; // Selesai
} else {
    // --- GAGAL BOOKING (Kamar Penuh) ---

    // Mutex hanya untuk print
    pthread_mutex_lock(&print_mutex);
    printf("%s ❌ Customer %02d gagal booking (percobaan %d), kamar penuh, mencoba
    lagi...\n",
           get_timestamp(), customer_id, attempts);
    pthread_mutex_unlock(&print_mutex);

    // Jika masih ada percobaan, tunggu 2 detik lalu coba lagi
    if (attempts < MAX_ATTEMPTS) {
        usleep(SLEEP_RETRY * 1000000);
    }
}

// --- GAGAL TOTAL ---
pthread_mutex_lock(&print_mutex);
printf("%s 💔 Customer %02d gagal total dan MENYERAH.\n",
       get_timestamp(), customer_id);
pthread_mutex_unlock(&print_mutex);

return NULL;
}

```

♦

```
// --- FUNGSI UTAMA ---
int main() {
    // Inisialisasi random seed
    srand(time(NULL));

    pthread_t threads[TOTAL_CUSTOMERS];
    int customer_ids[TOTAL_CUSTOMERS];

    // Inisialisasi Mutex
    if (pthread_mutex_init(&print_mutex, NULL) != 0) {
        printf("\nMutex init failed\n");
        return 1;
    }

    // Inisialisasi Semaphore (TOTAL_ROOMS = nilai awal)
    if (sem_init(&room_semaphore, 0, TOTAL_ROOMS) != 0) {
        printf("\nSemaphore init failed\n");
        return 1;
    }

    // Inisialisasi status kamar
    for(int i = 0; i < TOTAL_ROOMS; i++) {
        room_status[i] = 0; // Semua kamar kosong
    }

    printf("=====\n");
    printf(" 🏨 Simulasi Sistem Booking Hotel 🏨\n");
    printf("=====\n");
    printf("Total Kamar: %d | Total Customer: %d\n", TOTAL_ROOMS, TOTAL_CUSTOMERS);
    printf("-----\n");

    // Buat thread untuk setiap customer
    for (int i = 0; i < TOTAL_CUSTOMERS; i++) {
        customer_ids[i] = i + 1; // ID Customer 1 sampai 10
        if (pthread_create(&threads[i], NULL, customer_action, &customer_ids[i]) != 0) {
            perror("Error creating thread");
            return 1;
        }
    }

    // Tunggu semua thread selesai
    for (int i = 0; i < TOTAL_CUSTOMERS; i++) {
        pthread_join(threads[i], NULL);
    }

    // Hapus Semaphore dan Mutex
    sem_destroy(&room_semaphore);
    pthread_mutex_destroy(&print_mutex);
    printf("-----\n");
    printf("Simulasi Selesai. Semua customer telah check-out/menyerah.\n");
    printf("=====\n");
    return 0; }

```

Output :

```
chandrallinux@LAPTOP-E5CDVDCL:~$ nano hotel_booking.c
chandrallinux@LAPTOP-E5CDVDCL:~$ gcc hotel_booking.c -o hotel_booking -pthread
chandrallinux@LAPTOP-E5CDVDCL:~$ ./hotel_booking
=====
🏨 Simulasi Sistem Booking Hotel 🏨
=====
Total Kamar: 5 | Total Customer: 10
=====
[22:06:15] ⭐ Customer 02 mencoba booking...
[22:06:15] ⭐ Customer 04 mencoba booking...
[22:06:15] ⭐ Customer 07 mencoba booking...
[22:06:15] ⭐ Customer 08 mencoba booking...
[22:06:15] ⭐ Customer 06 mencoba booking...
[22:06:15] ⭐ Customer 01 mencoba booking...
[22:06:15] ⭐ Customer 03 mencoba booking...
[22:06:15] ⭐ Customer 05 mencoba booking...
[22:06:15] ⭐ Customer 09 mencoba booking...
[22:06:15] ⭐ Customer 10 mencoba booking...
[22:06:16] ✅ Customer 02 berhasil booking kamar 1! (4 kamar tersisa)
[22:06:16] ✅ Customer 08 berhasil booking kamar 2! (3 kamar tersisa)
[22:06:16] ✅ Customer 05 berhasil booking kamar 3! (2 kamar tersisa)
[22:06:17] ✅ Customer 04 berhasil booking kamar 4! (1 kamar tersisa)
[22:06:17] ✅ Customer 06 berhasil booking kamar 5! (0 kamar tersisa)
[22:06:17] ❌ Customer 03 gagal booking (percobaan 1), kamar penuh, mencoba lagi...
[22:06:17] ❌ Customer 10 gagal booking (percobaan 1), kamar penuh, mencoba lagi...
[22:06:18] ❌ Customer 07 gagal booking (percobaan 1), kamar penuh, mencoba lagi...
[22:06:18] ❌ Customer 01 gagal booking (percobaan 1), kamar penuh, mencoba lagi...
[22:06:18] ❌ Customer 09 gagal booking (percobaan 1), kamar penuh, mencoba lagi...
[22:06:19] ⭐ Customer 03 mencoba booking...
[22:06:19] ⭐ Customer 10 mencoba booking...
[22:06:20] ⭐ Customer 07 mencoba booking...
[22:06:20] ⭐ Customer 01 mencoba booking...
[22:06:20] ⭐ Customer 09 mencoba booking...
```

3. Cara Mengatasi Race Condition

Kami mengatasi *race condition* dengan dua mekanisme utama:

- **Semaphore (room_semaphore):** Mengatur **jumlah kamar** yang tersedia. Ini memastikan operasi *booking* (mengambil kamar) bersifat **atomik**, mencegah *overbooking* saat kamar menipis.
- **Mutex (print_mutex):** Melindungi **data bersama** (room_status) dan **semua output** (printf). Ini menjamin **integritas data** saat kamar ditandai terisi/kosong dan menjaga *output* konsol agar tidak kacau (tidak tumpang tindih).

4. Analisis Tanpa Sinkronisasi

Jika sinkronisasi tidak digunakan:

- **Tanpa Semaphore:** Terjadi **Overbooking**. Lebih dari 5 kamar akan diklaim terpakai karena *thread* secara bersamaan membaca data ketersediaan yang sama (misalnya, "1 kamar tersisa") dan mengklaimnya.
- **Tanpa Mutex:**
 - **Integritas Data Gagal:** Kamar yang sama bisa diklaim oleh dua *customer* (**klaim ganda**) karena *thread* menimpa status room_status satu sama lain.
 - **Output Kacau:** Log *booking* dan *check-out* akan **tumpang tindih** dan tidak dapat dibaca.

DINING PHILOSOPHERS PROBLEM (Akhiran NIM Genap)

Deskripsi Masalah:

Implementasikan solusi untuk masalah klasik 5 filsuf makan dengan **mencegah**

deadlock!

Kondisi:

- 5 filsuf duduk melingkar
- 5 sumpit (1 antara tiap filsuf)
- Filsuf perlu 2 sumpit untuk makan
- Filsuf berpikir → lapar → makan → berpikir (siklus)

Kode Program 1 Deadlock Demonstration:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

#define NUM_PHILOSOPHERS 5
#define EAT_TIME 2 // Waktu makan tetap agar deadlock lebih cepat

pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];

void *philosopher_deadlock(void *arg) {
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;

    while (1) {
        printf("[P%d] 🤔 Berpikir...\n", id);
        sleep(1); // Berpikir

        printf("[P%d] 🍴 Lapar, mencoba ambil sumpit...\n", id);

        // 1. Ambil Sumpit KIRI (Semua melakukan ini)
        pthread_mutex_lock(&chopsticks[left]);
        printf("[P%d] 🍴 Mengambil sumpit KIRI (%d).\n", id, left);

        // Mempersingkat waktu antara pengambilan sumpit untuk memaksimalkan deadlock
        usleep(100);

        // 2. Ambil Sumpit KANAN (Semua menunggu di sini)
        // Jika 5 filsuf berhasil mengambil kiri, mereka semua akan HANG di baris ini!
        printf("[P%d] 🤏 Menunggu sumpit KANAN (%d)...\n", id, right);
        pthread_mutex_lock(&chopsticks[right]);
```

```

    // Jika sampai sini, berarti berhasil makan
    printf("[P%d] 🍴 MAKAN...\n", id);
    sleep(EAT_TIME);

    // Selesai makan, letakkan kedua sumpit
    pthread_mutex_unlock(&chopsticks[right]);
    pthread_mutex_unlock(&chopsticks[left]);
    printf("[P%d] 🍴 Menaruh kedua sumpit.\n", id);
}
return NULL;
}

int main_deadlock() {
    pthread_t philosophers[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];

    printf("=== 💀 DEADLOCK DEMONSTRATION ===\n");
    printf("Semua filsuf akan mengambil KIRI, lalu KANAN.\n");
    printf("-----\n");

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_init(&chopsticks[i], NULL);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        pthread_create(&philosophers[i], NULL, philosopher_deadlock, &ids[i]);
    }

    // Biarkan program berjalan hingga hang
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(philosophers[i], NULL);
    }

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&chopsticks[i]);
    }
    return 0;
}
// Untuk menjalankan: panggil main_deadlock() dari main()

```

```

chandrinux@LAPTOP-E5CDVDCL:~$ nano deadlock_demo.c
chandrinux@LAPTOP-E5CDVDCL:~$ gcc deadlock_demo.c -o deadlock_demo -pthread
chandrinux@LAPTOP-E5CDVDCL:~$ ./deadlock_demo
=== 💀 DEADLOCK DEMONSTRATION ===
Semua filsuf akan mengambil KIRI, lalu KANAN.
-----
[P1] 🤔 Berpikir...
[P3] 🤔 Berpikir...
[P0] 🤔 Berpikir...
[P2] 🤔 Berpikir...
[P4] 🤔 Berpikir...
[P1] 🍴Lapar, mencoba ambil sumpit...
[P1] // Mengambil sumpit KIRI (1).
[P3] 🍴Lapar, mencoba ambil sumpit...
[P3] // Mengambil sumpit KIRI (3).
[P2] 🍴Lapar, mencoba ambil sumpit...
[P2] // Mengambil sumpit KIRI (2).
[P0] 🍴Lapar, mencoba ambil sumpit...
[P0] // Mengambil sumpit KIRI (0).
[P1] 🤚 Menunggu sumpit KANAN (2)...
[P4] 🍴Lapar, mencoba ambil sumpit...
[P4] // Mengambil sumpit KIRI (4).
[P3] 🤚 Menunggu sumpit KANAN (4)...
[P2] 🤚 Menunggu sumpit KANAN (3)...
[P0] 🤚 Menunggu sumpit KANAN (1)...
[P4] 🤚 Menunggu sumpit KANAN (0)...
^C
chandrinux@LAPTOP-E5CDVDCL:~$ |

```

Kode Program 2 Deadlock Prevention:

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>
#include <string.h>
#include <semaphore.h>

#define NUM_PHILOSOPHERS 5
#define RUN_TIME 60 // Waktu simulasi dalam detik
#define THINK_TIME 1
#define EAT_TIME 2

// --- GLOBAL VARIABLES for Synchronization & Analysis ---
pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];
sem_t room_semaphore; // Untuk Solusi B: Limiting Philosophers
long long eat_counts[NUM_PHILOSOPHERS];
time_t start_time;
pthread_mutex_t count_mutex; // Untuk melindungi eat_counts

// --- UTILITY FUNCTIONS ---

// Fungsi untuk format waktu [MM:SS]
void print_timestamp() {

```

```

time_t current_time = time(NULL);
double elapsed = difftime(current_time, start_time);
int minutes = (int)elapsed / 60;
int seconds = (int)elapsed % 60;

// Gunakan count_mutex untuk memastikan printf tidak kacau
pthread_mutex_lock(&count_mutex);
printf("[%02d:%02d]", minutes, seconds);
pthread_mutex_unlock(&count_mutex);
}

// --- SOLUSI A: Resource Ordering (Ganjil/Genap) ---
void *philosopher_ordering(void *arg) {
    int id = *(int *)arg;
    int first_chopstick, second_chopstick;

    // Filsuf Genap (0, 2, 4): Kiri -> Kanan (Lower Index -> Higher Index)
    if (id % 2 == 0) {
        first_chopstick = id;
        second_chopstick = (id + 1) % NUM_PHILOSOPHERS;
    }
    // Filsuf Ganjil (1, 3): Kanan -> Kiri (Higher Index -> Lower Index)
    else {
        first_chopstick = (id + 1) % NUM_PHILOSOPHERS;
        second_chopstick = id;
    }

    while (difftime(time(NULL), start_time) < RUN_TIME) {
        // 1. Berpikir
        print_timestamp(); printf(" 🤔 Filsuf-%d sedang berpikir...\n", id + 1);
        sleep(THINK_TIME);

        // 2. Lapar, mencoba ambil sumpit
        print_timestamp(); printf(" 🍴 Filsuf-%d lapar, mencoba ambil sumpit...\n", id + 1);

        // Ambil sumpit pertama
        pthread_mutex_lock(&chopsticks[first_chopstick]);
        print_timestamp(); printf(" 🍴 Filsuf-%d mengambil sumpit pertama (%d).\n", id + 1,
first_chopstick);

        // Ambil sumpit kedua
        pthread_mutex_lock(&chopsticks[second_chopstick]);
        print_timestamp(); printf(" 🍴 Filsuf-%d mengambil sumpit kedua (%d).\n", id + 1,
second_chopstick);

        // 3. Makan
        print_timestamp(); printf(" 🍽️ Filsuf-%d sedang MAKAN...\n", id + 1);
        sleep(EAT_TIME);

        // 4. Meletakkan sumpit dan menghitung
        pthread_mutex_unlock(&chopsticks[first_chopstick]);
        pthread_mutex_unlock(&chopsticks[second_chopstick]);

        pthread_mutex_lock(&count_mutex);

```

♦

```

    eat_counts[id]++;
    pthread_mutex_unlock(&count_mutex);

    print_timestamp(); printf(" □ Filsuf-%d menaruh kedua sumpit.\n", id + 1);
}
return NULL;
}

// --- SOLUSI B: Limiting Philosophers (Max 4 di Ruangan) ---
void *philosopher_limiting(void *arg) {
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;

    while (difftime(time(NULL), start_time) < RUN_TIME) {
        // 1. Berpikir
        print_timestamp(); printf(" ☹ Filsuf-%d sedang berpikir...\n", id + 1);
        sleep(THINK_TIME);

        // 2. Lapar: Coba masuk ke 'ruangan' (ambil semaphore)
        print_timestamp(); printf(" 🍴 Filsuf-%d mencoba masuk ruang makan...\n", id + 1);
        sem_wait(&room_semaphore);

        // 3. Ambil Sumpit (Urutan standar: Kiri -> Kanan. Aman karena max 4)
        print_timestamp(); printf(" 🍴 Filsuf-%d lapar, mengambil sumpit...\n", id + 1);

        pthread_mutex_lock(&chopsticks[left]);
        print_timestamp(); printf(" □ Filsuf-%d mengambil sumpit KIRI (%d).\n", id + 1, left);

        pthread_mutex_lock(&chopsticks[right]);
        print_timestamp(); printf(" □ Filsuf-%d mengambil sumpit KANAN (%d).\n", id + 1, right);

        // 4. Makan
        print_timestamp(); printf(" 😋 Filsuf-%d sedang MAKAN...\n", id + 1);
        sleep(EAT_TIME);

        // 5. Meletakkan sumpit, hitung, dan keluar 'ruangan'
        pthread_mutex_unlock(&chopsticks[left]);
        pthread_mutex_unlock(&chopsticks[right]);

        pthread_mutex_lock(&count_mutex);
        eat_counts[id]++;
        pthread_mutex_unlock(&count_mutex);

        print_timestamp(); printf(" □ Filsuf-%d menaruh kedua sumpit.\n", id + 1);
        sem_post(&room_semaphore); // Keluar ruangan
    }
    return NULL;
}

// --- MAIN CONTROL FUNCTION ---
void run_simulation(void (*func)(void *), const char *solution_name) {
    pthread_t philosophers[NUM_PHILOSOPHERS];

```

♦

```

int ids[NUM_PHILOSOPHERS];

printf("\n\n=== DINING PHILOSOPHERS PROBLEM ===\n");
printf("Solusi: %s\n", solution_name);
printf("-----\n");

// Reset dan inialisasi untuk simulasi baru
memset(eat_counts, 0, sizeof(eat_counts));
for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
    pthread_mutex_init(&chopsticks[i], NULL);
}
pthread_mutex_init(&count_mutex, NULL);

if (strcmp(solution_name, "Limiting Philosophers (Max 4)") == 0) {
    // Inisialisasi Semaphore dengan nilai 4 (N-1)
    sem_init(&room_semaphore, 0, NUM_PHILOSOPHERS - 1);
}

start_time = time(NULL);

// Buat thread
for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
    ids[i] = i;
    pthread_create(&philosophers[i], NULL, func, &ids[i]);
}

// Tunggu sampai RUN_TIME selesai
sleep(RUN_TIME);

// Kirim sinyal cancel ke semua thread (agar tidak menunggu join)
for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
    pthread_cancel(philosophers[i]);
}

// Cleanup
for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
    pthread_mutex_destroy(&chopsticks[i]);
}
pthread_mutex_destroy(&count_mutex);
if (strcmp(solution_name, "Limiting Philosophers (Max 4)") == 0) {
    sem_destroy(&room_semaphore);
}
}

int main() {
    // 1. RUN DEADLOCK DEMONSTRATION (UNCOMMENT JIKA INGIN MENGHANGKAN PROGRAM)
    // main_deadlock(); // Program akan hang di sini

    // 2. RUN DEADLOCK PREVENTION SOLUSI A
    run_simulation(philosopher_ordering, "Resource Ordering (Ganjil/Genap)");

    // 3. ANALISIS SOLUSI A
    printf("\n\n=== STATISTIK (60 detik) ===\n");
    long long total_eats_A = 0;

```

♦

```

for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
    printf("Filsuf-%d: Makan %lld kali\n", i + 1, eat_counts[i]);
    total_eats_A += eat_counts[i];
}
printf("Total Makan: %lld\n", total_eats_A);
printf("✅ Tidak ada deadlock!\n");

// 4. RUN DEADLOCK PREVENTION SOLUSI B
run_simulation(philosopher_limiting, "Limiting Philosophers (Max 4)");

// 5. ANALISIS SOLUSI B
printf("\n\n=== STATISTIK (60 detik) ===\n");
long long total_eats_B = 0;
for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
    printf("Filsuf-%d: Makan %lld kali\n", i + 1, eat_counts[i]);
    total_eats_B += eat_counts[i];
}
printf("Total Makan: %lld\n", total_eats_B);
printf("✅ Tidak ada deadlock!\n");

return 0;
}

```

```

chandrinux@LAPTOP-E5CDVDCL:~$ nano dining_philosophers.c
chandrinux@LAPTOP-E5CDVDCL:~$ gcc dining_philosophers.c -o dining_philosophers -pthread
chandrinux@LAPTOP-E5CDVDCL:~$ ./dining_philosophers

=== DINING PHILOSOPHERS PROBLEM ===
Solusi: Resource Ordering (Ganjil/Genap)
-----
[00:00] 🤖 Filsuf-1 sedang berpikir...
[00:00] 🤖 Filsuf-2 sedang berpikir...
[00:00] 🤖 Filsuf-3 sedang berpikir...
[00:00] 🤖 Filsuf-4 sedang berpikir...
[00:00] 🤖 Filsuf-5 sedang berpikir...
[00:01] 🍴 Filsuf-2 lapar, mencoba ambil sumpit...
[00:01] // Filsuf-2 mengambil sumpit pertama (2).
[00:01] // Filsuf-2 mengambil sumpit kedua (1).
[00:01] 🍴 Filsuf-2 sedang MAKAN...
[00:01] 🍴 Filsuf-4 lapar, mencoba ambil sumpit...
[00:01] // Filsuf-4 mengambil sumpit pertama (4).
[00:01] // Filsuf-4 mengambil sumpit kedua (3).
[00:01] 🍴 Filsuf-4 sedang MAKAN...
[00:01] 🍴 Filsuf-3 lapar, mencoba ambil sumpit...
[00:01] 🍴 Filsuf-1 lapar, mencoba ambil sumpit...
[00:01] // Filsuf-1 mengambil sumpit pertama (0).
[00:01] 🍴 Filsuf-5 lapar, mencoba ambil sumpit...
[00:03] // Filsuf-2 menaruh kedua sumpit.
[00:03] 🤖 Filsuf-2 sedang berpikir...
[00:03] // Filsuf-3 mengambil sumpit pertama (2).
[00:03] // Filsuf-4 menaruh kedua sumpit.
[00:03] 🤖 Filsuf-4 sedang berpikir...
[00:03] // Filsuf-5 mengambil sumpit pertama (4).
[00:03] // Filsuf-3 mengambil sumpit kedua (3).
[00:03] 🍴 Filsuf-3 sedang MAKAN...

```

3. Analisis Perbandingan Efisiensi

Metrik	Solusi A: Resource Ordering	Solusi B: Limiting Philosophers
Pencegahan Deadlock	Terjamin. Memecah <i>circular wait</i> .	Terjamin. Memecah <i>hold and wait</i> pada resource terakhir.
Total Makan (Throughput)	Lebih Tinggi (Misalnya: 35-40 kali)	Lebih Rendah (Misalnya: 25-30 kali)
Distribusi Fairness	Baik. Setiap filsuf cenderung memiliki kesempatan yang sama, tetapi pola pengambilan yang berbeda bisa sedikit memengaruhi.	Sangat Baik. Hanya 4 yang bersaing untuk sumpit; semua memiliki akses yang sama ke 4 sumpit pertama.
Implementasi	Lebih kompleks karena membutuhkan logika pengecekan ID filsuf.	Lebih sederhana (hanya perlu Semaphore tambahan).
Kelemahan	Risiko starvation pada filsuf yang selalu gagal memenangkan persaingan sumpit.	Menurunkan Throughput karena membatasi konkurensi (hanya 4 yang boleh mencoba, padahal 5 bisa makan berturut-turut jika urutannya tepat).

Analisis mendalam:

- Mengapa deadlock bisa terjadi?
- Bagaimana masing-masing solusi bekerja?
- Solusi mana yang paling adil (fair)?
- Apakah ada kemungkinan starvation?

1. Mengapa Deadlock Terjadi?

Deadlock terjadi karena terpenuhinya empat syarat kunci (Kondisi Coffman), terutama

Menunggu Melingkar (Circular Wait).

Dalam skenario awal, semua 5 filsuf:

- 1. Berhasil mengambil **sumpit kiri** mereka.
- 2. Semuanya **menunggu sumpit kanan** mereka, yang dipegang oleh tetangga di sebelah kanan.

Ini menciptakan lingkaran tertutup: P1 menunggu P2, P2 menunggu P3, P5 menunggu P1. Karena tidak ada yang mau melepaskan sumpit yang sudah dipegang, semua proses berhenti (**HANG**).

2. Cara Kerja Masing-Masing Solusi

Tujuan dari setiap solusi adalah **mematahkan** salah satu dari empat syarat *deadlock*.

Solusi	Kondisi yang Dipatahkan	Bagaimana Cara Kerjanya?
A: Resource Ordering (Ganjil/Genap)	Menunggu Melingkar	Memaksa satu filsuf (misalnya, yang ID-nya ganjil) untuk mengambil sumpit dalam urutan yang berbeda (Kanan $\rightarrow$ Kiri), sementara yang lain Kiri $\rightarrow$ Kanan. Ini menghancurkan rantai tunggu melingkar.
B: Limiting Philosophers (Max 4)	Menahan dan Menunggu	Menggunakan Semaphore untuk membatasi hanya 4 filsuf yang boleh masuk ruang makan. Ini menjamin bahwa selalu ada setidaknya satu sumpit gratis untuk filsuf terakhir yang masuk, memungkinkannya mengambil dua sumpit dan makan.

3. Keadilan (Fairness) dan Starvation

Metrik	Solusi A (Ordering)	Solusi B (Limiting)
Paling Adil (Fair)	Cenderung Kurang Adil. Rentan terhadap Starvation karena filsuf yang memiliki pola pengambilan yang sama mungkin secara konsisten kalah dalam persaingan.	Paling Adil. Menggunakan Semaphore (yang biasanya FIFO) untuk masuk, memastikan <i>thread</i> yang paling lama menunggu akan mendapat giliran masuk dan makan.
Kemungkinan Starvation	Tinggi. Filsuf yang berulang kali kalah dalam persaingan untuk sumpit yang sama dapat tidak pernah mendapat giliran makan.	Rendah. Begitu seorang filsuf diizinkan masuk (mendapatkan Semaphore), ia dijamin dapat mengambil kedua sumpitnya, mencegah kelaparan jangka panjang.

♦